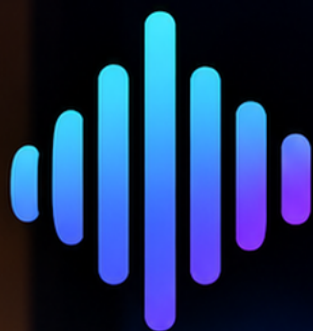




MTA

AUDIO EDITOR

Specifica completa del formato MTA (Multi Track Audio)

Struttura on-disk, stream audio, timing, metadata e validazione

Versione: 0.2.0 - Revision: 157

Build: 20261006010205

Alessandro De Salvo

braket71@gmail.com

<https://github.com/desalvo/mta-audio-editor>

Licenza: EUPL-1.2

Digital Audio Workstation with MTA support



Windows



macOS



Linux



Android



iOS

Indice

1. Ambito e livello di validazione

2. Organizzazione generale

3. Trasporto media MTA

3.1 Trasformazione

3.2 Validazione sul corpus

4. Primitive EBML/Matroska

5. Audio

6. SimpleBlock

7. Cluster e timebase

8. Cues

9. SeekHead

10. Primo frame Xing/Info

10.1 TOC

10.2 CRC

11. Attachment

12. Attachment SYL

12.1 Keystream condivisa da 256 byte

12.2 Prefisso binario comune

13. COLORS

13.1 Posizione 127

14. LYRICS e CHORDS

15. MIDITK

16. MtxInfoData XML

17. Semantica NoteOn

18. PreCntUSec

19. Ruoli Click e Melody

20. Algoritmo di lettura

21. Algoritmo di scrittura

22. Livelli di compatibilità

23. Invarianti di validazione

24. Limiti e collaudo

Formato MTA (Multi Track Audio) - specifica tecnica consolidata

Versione documento: 0.2.0-93

1. Ambito e livello di validazione

In questa documentazione **MTA** indica **Multi Track Audio**, il formato multitraccia usato per mantenere separati stream musicali, ruoli e dati temporizzati.

Questa specifica descrive il formato MTA su disco e le regole di lettura/scrittura validate sul corpus MTA disponibile al progetto. Distingue la struttura byte-level, la semantica dei campi, la politica di authoring e la validazione finale sul dispositivo/firmware target. Il corpus disponibile contiene file indipendenti con 10-14 stream audio; i file musicali originali non vengono distribuiti con il progetto.

2. Organizzazione generale

MTA usa un contenitore derivato da Matroska/EBML. La parte iniziale contiene strutture EBML leggibili normalmente:

```
EBML Header
Segment
  SeekHead
  Void
  Info
  Tracks
  Attachments
  Cues
  Cluster ... trasporto media MTA
  Cluster ...
```

L'offset dati del Segment è il primo byte dopo ID e VINT della dimensione. SeekPosition e CueClusterPosition sono relativi a questo punto. Dal primo byte del primo Cluster fino a EOF viene applicato il trasporto media descritto nella sezione successiva.

3. Trasporto media MTA

3.1 Trasformazione

La regione media usa uno XOR periodico di 984 byte:

```
plain[i]  = stored[i] XOR key[i mod 984]
stored[i] = plain[i]   XOR key[i mod 984]
```

con $i = 0$ sul primo byte del primo Cluster. La fase parte dal primo byte dell'ID Cluster, continua fino a EOF, non si resetta ai confini Cluster/Cue ed è simmetrica in lettura e scrittura.

Identità della chiave documentata:

```
periodo: 984 byte
SHA-256: bcb30443707bdc8b651c1a58aa4152438ce5a6632cc98b294501eb08adbb3547
```

3.2 Validazione sul corpus

La periodicità è verificabile attraverso plaintext Matroska prevedibile: ID Cluster 1F 43 B6 75, Timecode, SimpleBlock, VINT del numero traccia, timecode relativi, flag e header MPEG. Applicando il periodo documentato dal primo Cluster a EOF, i campioni validati diventano media Matroska canonica leggibile da FFmpeg per l'intera regione media.

4. Primitive EBML/Matroska

Elemento	ID esadecimale
EBML	1A45DFA3
Segment	18538067
SeekHead	114D9B74
Info	1549A966
Tracks	1654AE6B
Attachments	1941A469
Cues	1C53BB6B
Cluster	1F43B675
Timecode	E7
SimpleBlock	A3
CuePoint	BB
CueTime	B3
CueTrackPositions	B7
CueClusterPosition	F1

Nel corpus il Segment copre esattamente il resto del file fino a EOF.

5. Audio

Gli stream documentati sono:

```
MPEG-1 Layer III
sample rate: 44100 Hz
canali: stereo
bitrate nominale: 320000 bit/s
```

Ogni SimpleBlock trasporta un singolo frame MP3. Per 320 kbit/s a 44.1 kHz:

```
base = floor(144 * 320000 / 44100) = 1044 byte
```

I frame misurano quindi 1044 o 1045 byte in base al padding. Il pacchetto zero è un frame Info da 1044 byte senza padding.

Lo schedule del padding successivo è riproducibile con:

```
padding = [0]
state = 4
for each later musical frame:
    state += 44
    if state >= 49:
        padding.append(1)
        state -= 49
    else:
        padding.append(0)
```

6. SimpleBlock

Layout tipico:

```
A3
<size VINT, normalmente 2 byte>
<track number VINT, 1 byte>
<relative timecode int16 big-endian>
80
<MP3 frame>
```

L'overhead tipico è 7 byte. I blocchi sono ordinati frame-major: per ogni istante vengono scritte in sequenza tutte le tracce.

7. Cluster e timebase

Un Cluster completo contiene dieci frame-time per ogni stream. I timecode relativi sono:

```
0, 144, 288, 432, 576, 720, 864, 1008, 1152, 1296
```

Valori Matroska validati:

```
TimecodeScale    = 181392 ns
DefaultDuration  = 26122448 ns
```

Il timecode assoluto del Cluster segue:

```
cluster_timecode(i) = floor(i * (8000000000 / 555513) + 1/16)
```

L'ultimo Cluster può avere meno di dieci frame-time e usa il VINT EBML minimo necessario alla dimensione reale.

8. Cues

Viene emesso un Cue ogni quattro Cluster, cioè ogni 40 pacchetti MP3 per traccia:

```
cue_count = ceil(packet_count_per_track / 40)
Cue n -> Cluster 4*n
```

CueClusterPosition è relativo all'inizio dati del Segment. Tempi e posizioni sono determinabili da conteggio pacchetti, schedule frame, numero tracce e dimensioni Cluster.

9. SeekHead

Il SeekHead riferisce le strutture principali pre-media e il primo Cluster. Le posizioni sono Segment-relative. Poiché la larghezza minima degli interi EBML può cambiare quando gli offset superano una soglia, il writer deve risolvere il layout con iterazione a punto fisso: assumere le larghezze, calcolare lunghezze/offset, ricalcolare le larghezze minime e ripetere fino a stabilità.

10. Primo frame Xing/Info

Il primo pacchetto di ogni traccia è un frame MP3 Info non padded da 1044 byte.

Offset	Campo
0..3	header MPEG
36..39	ASCII Info
40..43	flag 0x0000000f
44..47	frame musicali, Info escluso
48..51	byte MP3 totali, Info incluso
52..151	Xing TOC, 100 byte
152..155	qualità
156..	stringa encoder
177..179	delay + padding, 12 bit + 12 bit
184..187	music length
188..189	music CRC
190..191	tag CRC

Relazioni:

```
Info.frames = total_packets - 1
Info.bytes = somma di tutti i frame MP3, incluso Info
music_length = Info.bytes
```

10.1 TOC

La TOC a 100 byte segue l'algoritmo FFmpeg/LAME basato su un bag di 400 entry e campionamento in 100 valori.

10.2 CRC

Entrambi i CRC usano CRC-16/ANSI riflesso con polinomio 0xA001 e valore iniziale zero. Il music CRC copre tutti i frame musicali in ordine, escludendo Info; il tag CRC copre i byte 0..189 del frame Info dopo l'inserimento del music CRC.

11. Attachment

Nei file validati compaiono quattro attachment tipici:

```
SYL originale  
SYL modificato  
MtxInfoData XML originale  
MtxInfoData XML modificato
```

Nel corpus ispezionato le copie original/mod sono byte-identiche. MTA Audio Editor preserva gli attachment importati byte-per-byte quando possibile. Se l'estrazione standard non riesce, il parser compatibilità può recuperare FileData usando le dimensioni del container.

12. Attachment SYL

Il file SYL è un tag ID3v2.3 completo con sezioni M-Live aggiuntive:

```
DELTA  
LYRICS  
COLORS  
MIDITK  
CHORDS
```

I frame ID3 standard possono contenere titolo, artista, BPM, tonalità, durata, publisher, compositore, genere, copyright, anno, album e ISRC.

12.1 Keystream condivisa da 256 byte

LYRICS, COLORS, MIDITK e CHORDS usano una stessa sequenza XOR da 256 byte. La sequenza validata è stabile nei campioni e contiene una permutazione dei valori 00..FF.

12.2 Prefisso binario comune

I body delle sezioni documentate iniziano con:

```
28 B6 A9
```

prima del payload della sezione.

13. COLORS

I record fisici normali COLORS sono lunghi 15 byte. Dopo la trasformazione il layout validato è:

```
LF d d RS d d 'm' p p p '=' ':' 'k' NUL next_minute
```

Le cifre decimali codificano:

```
centiseconds = d1*1000 + d2*100 + d3*10 + d4
position      = p1*100 + p2*10 + p3
time_ms       = current_minute*60000 + centiseconds*10
```

Fase di trasformazione:

```
key_index = (record_index - 17*byte_index) mod 256
stored = plain XOR key[key_index] XOR 0x0A
```

L'ultimo record fisico è un terminatore distinto, non un evento normale.

13.1 Posizione 127

Il valore 127 è un controllo di reset/cambio pagina della visualizzazione, non una posizione carattere ordinaria. Nel corpus segue l'ultimo highlight progressivo della pagina precedente e precede il primo evento della nuova pagina.

14. LYRICS e CHORDS

Dopo il prefisso a tre byte, queste sezioni usano una fase sul payload piatto:

```
key_index = 239 * flat_payload_offset mod 256
stored = plain XOR key[key_index] XOR 0x0A
```

I byte di testo/accordo usano inoltre:

```
text_byte ^= 0x30
```

I record terminano con il marker documentato =:k\0X dopo decodifica. Il timing usa minuti più quattro cifre di centesimi, come COLORS.

15. MIDITK

MIDITK usa:

```
key_index = 239 * (payload_offset - 3) mod 256
stored = midi_byte XOR key[key_index] XOR 0x0A XOR 0x30
```

La lettura produce uno Standard MIDI File valido. Caratteristiche validate:


```
SMF format 0
un solo MTrk
PPQ 480
meta-event tempo
meta-event marker
meta-event end-of-track
```

I marker includono sezioni come Intro, Verse, Chorus, Bridge, Coda e Fine.

16. MtxInfoData XML

L'attachment XML ha root MtxInfoData. Gruppi logici osservati:

```
Identity
General
TrkAudio / Track
```

I campi includono identità utente/applicazione, tonalità base, BPM base, durata, pre-count in microsecondi, numero/nome/tipo traccia, routing, volume, mute/solo e NoteOn. Alcuni payload contengono zero padding terminale, preservato come parte dell'attachment.

17. Semantica NoteOn

NoteOn è un vettore 0/1 separato da punto e virgola, con circa una decisione di attività per secondo e per traccia. Sul corpus analizzato:

```
4 brani
50 tracce
12.654 bin da un secondo
36/50 tracce riprodotte esattamente da una migliore soglia RMS singola
45/50 differiscono al massimo di un bin
21/12654 differenze totali = 0,166%
```

Le differenze residue si concentrano su onset, release, leakage/overlap codec e materiale sostenuto molto debole. Questo supporta l'interpretazione di NoteOn come maschera di attività della sorgente/pre-encode, non come semplice soglia fissa calcolata sull'MP3 finale.

Politica di authoring:

- MTA importato: preservare NoteOn originale;
- progetto nuovo: derivare l'attività a un secondo dalla timeline sorgente/pre-encode.

18. PreCntUSec

PreCntUSec è una durata esplicita di pre-count/lead-in in microsecondi. E' correlata al lead-in musicale ma non coincide sempre con una formula semplice a quattro beat calcolata dal BaseBpm arrotondato; va quindi trattata come proprietà temporale esplicita.

19. Ruoli Click e Melody

I ruoli vengono mantenuti separati dagli stem ordinari e applicati tramite un profilo dispositivo in export.

Profilo	Click	Melody
Merish5 / Xynthia2 MTA8	8	7
B.Beat / B.Beat Plus / Evo / Pro16 / DIVO MTA8	7	8
M-Live MTA16, default corpus	1	9

Per MTA16, il default deriva dai file disponibili: Metronome è stream 1 in ogni campione osservato; Melody Track appare più spesso nello stream 9 e in un campione nello stream 10. Lo stream 9 è quindi un default pratico e modificabile. Quando serve preservare una posizione fisica non contigua, il writer inserisce stream silenziosi intermedi.

20. Algoritmo di lettura

1. analizzare l'area EBML leggibile pre-media;
2. trovare il primo Cluster attraverso SeekHead/Cues;
3. controllare i byte all'offset;
4. se è presente 1F43B675, trattare il media come Matroska canonico;
5. altrimenti applicare il periodo 984-byte a un probe;
6. se il probe produce 1F43B675, trasformare l'intera regione media in una vista temporanea canonica;
7. usare FFmpeg/FFprobe per l'estrazione media;
8. preservare separatamente i byte originali degli attachment;
9. interpretare SYL/XML solo quando gli invarianti strutturali validano;
10. conservare i campi non modificati necessari al round-trip.

21. Algoritmo di scrittura

1. renderizzare ogni slot di output;
2. applicare mapping e merge delle tracce assegnate allo stesso slot;
3. creare eventuali stream silenziosi per preservare posizioni fisiche;
4. muxare una rappresentazione Matroska canonica;
5. preservare o creare gli attachment necessari;
6. risolvere Seek/Cue/offset e larghezze EBML minime;
7. localizzare il primo Cluster canonico;
8. applicare la trasformazione da quel Cluster a EOF;
9. verificare che la trasformazione inversa ripristini il Cluster canonico;
10. validare con FFprobe e con gli invarianti MTA principali.

Un writer deterministico può inoltre usare le formule esatte di pacchetti, Cluster, Cue, SeekHead e Xing/Info documentate qui, anziché delegare ogni dettaglio a un muxer generico.

22. Livelli di compatibilità

Livello	Significato
Lettura strutturale	container, stream e attachment recuperabili
Lettura estesa	trasporto media, SYL e MtxInfoData interpretati
Scrittura strutturale	Matroska valido più trasporto MTA
Scrittura conservativa	attachment importati preservati byte-per-byte
Authoring metadata	nuovi SYL/XML generati secondo la semantica documentata
Accettazione hardware	file collaudato su specifico dispositivo/firmware

I primi livelli sono automatizzabili in CI; l'accettazione hardware richiede il target fisico o un'implementazione vendor affidabile.

23. Invarianti di validazione

Un MTA generato deve verificare almeno:

```
EBML header presente
Segment presente
primo Cluster identificabile
forma media MTA riconosciuta
primo Cluster canonico dopo trasformazione
round-trip della trasformazione byte-identico
media canonica leggibile da FFprobe
numero stream coerente con output e capacità
attachment estraibili e non corrotti
Click/Melody nelle posizioni fisiche del profilo
durata e timing coerenti
```

Per writer deterministici verificare inoltre frame MP3, padding, overhead SimpleBlock, ordine frame-major, timecode/span Cluster, Cue, Info.frames, Info.bytes, TOC, CRC, Segment size ed EOF alignment.

24. Limiti e collaudo

La specifica documenta le strutture necessarie per leggere e scrivere il formato validato. Differenze di firmware o di strumenti di authoring possono usare scelte differenti ma compatibili. Prima dell'uso live un file esportato va sempre collaudato sul dispositivo/firmware target. Non va promessa una riproduzione bit-identica di ogni versione encoder se non confrontata con un riferimento controllato generato da quella stessa versione.